UTN
Facultad Regional Córdoba

TUP

Tecnicatura Universitaria en Programación

BASES DE DATOS II

Unidad Temática N°3:
Bases de Datos Orientada a Grafos

Teórico
2° Año – 3° Cuatrimestre

CC BY ND NC

Introducción a las bases de datos basadas en grafos **3**

Componenetes de una bases de datos basadas en grafos. 3

Ventajas de trabajar con bases de datos de grafos 4

Diferencias con otros tipos de bases de datos 4

Casos de uso 5

Introducción a Neo4j **6**

Introducción al lenguaje Cypher 6

BIBLIOGRAFÍA **8**

Introducción a las bases de datos basadas en grafos

Las bases de datos de grafos han surgido como una alternativa a los modelos relacionales y NoSQL tradicionales, proporcionando una forma más intuitiva y eficiente de modelar relaciones complejas entre datos. En este contexto, Neo4j se ha consolidado como una de las principales bases de datos de grafos, permitiendo representar datos en forma de nodos y relaciones con propiedades asociadas.

Neo4j utiliza un modelo basado en grafos dirigidos y propiedades (Labeled Property Graph Model), lo que lo hace ideal para aplicaciones donde las conexiones entre los datos son fundamentales. Su motor de consulta, basado en el lenguaje **Cypher**, facilita la manipulación y recuperación de información de manera eficiente.

Neo4j es altamente escalable y permite el procesamiento eficiente de grandes volúmenes de datos conectados. Su diseño basado en grafos es óptimo para problemas que involucran estructuras complejas de interconexión, como la detección de comunidades en redes sociales o la optimización de rutas en sistemas de logística.

Componenetes de una bases de datos basadas en grafos.

Una base de datos de grafos está compuesta por:

- **Nodos**: Representan entidades u objetos dentro del dominio del problema. Cada nodo puede tener una o más etiquetas para clasificarlo.
- **Relaciones:** Conectan nodos y definen cómo están vinculados entre sí. Son direccionales y pueden tener propiedades asociadas.
- **Propiedades**: Son pares clave-valor que pueden ser almacenados en nodos y relaciones, proporcionando información adicional sobre estos.

Ejemplo de una estructura de grafo para una red social:

- **Nodos**: *Usuario, Publicación*
- **Relación**: *SIGUE, CREÓ*
- **Propiedades de Usuario**: *nombre, edad, ubicación*
- **Propiedades de Publicación**: *contenido, fecha, categoría*

Cada relación puede incluir propiedades adicionales, como la fecha en que se estableció la conexión entre usuarios o el nivel de interacción entre ellos.

Ventajas de trabajar con bases de datos de grafos

Las principales ventajas que encontramos a la hora de trabajar con Base de Grafos son las siguientes:

Eficiencia en consultas complejas: Los modelos relacionales requieren múltiples uniones (JOINs) para consultar relaciones entre datos, lo que puede ser costoso en términos de rendimiento. En cambio, las bases de datos de grafos permiten recuperar relaciones de manera directa sin necesidad de operaciones de combinación.

Escalabilidad: Se pueden distribuir los datos eficientemente en diferentes servidores sin perder rendimiento en las consultas.

Flexibilidad en la modelación de datos: A diferencia de las bases de datos relacionales que requieren esquemas rígidos, las bases de datos de grafos permiten agregar nuevas relaciones y propiedades sin afectar el rendimiento.

Ideal para análisis de redes: Su diseño facilita el análisis de redes sociales, recomendaciones, detección de fraudes y otras aplicaciones donde las conexiones entre los datos son esenciales.

Optimización de algoritmos basados en grafos: Algoritmos como PageRank, detección de comunidades y búsqueda de caminos más cortos funcionan de manera eficiente en bases de datos de grafos.

Diferencias con otros tipos de bases de datos

Las bases de datos de grafos presentan diferencias significativas con otros modelos como las bases de datos relacionales y NoSQL.

En comparación con las bases de datos relacionales, las bases de datos de grafos no utilizan tablas para almacenar datos. Mientras que en una base de datos relacional la información se organiza en filas y columnas con estructuras fijas, en una base de datos de grafos los datos se representan como nodos y relaciones, lo que permite una mayor flexibilidad y rapidez en consultas donde las conexiones son clave. Además, en los sistemas relacionales, las consultas que involucran múltiples relaciones requieren el uso de JOINs, lo que puede afectar el rendimiento, mientras que en las bases de datos de grafos este tipo de consultas se resuelve de manera más eficiente mediante la navegación por los nodos.

Por otro lado, en comparación con las bases de datos NoSQL basadas en documentos o clave-valor, las bases de datos de grafos se centran en la interconexión de datos. Las bases de datos NoSQL son ideales para almacenar grandes volúmenes de información no estructurada y permiten una rápida recuperación de datos específicos, pero no son tan eficientes cuando se necesita analizar y gestionar múltiples relaciones entre los datos. En cambio, las bases de datos de grafos permiten modelar estas relaciones de forma directa, sin la necesidad de replicar información en distintos documentos o estructuras de almacenamiento.

Estas diferencias hacen que las bases de datos de grafos sean la mejor opción para aplicaciones donde la estructura y la interconexión de los datos son fundamentales.

Comparación breve con el modelo relacional:

Relacional	**Grafos**
Tablas	Nodos
Claves foráneas	Relaciones (edges)
JOINs	Caminos (paths)
SQL	Cypher

Casos de uso

Las bases de datos de grafos son especialmente útiles en aplicaciones donde las relaciones son críticas. Algunos ejemplos incluyen:

Redes Sociales: Representación de conexiones entre usuarios, publicaciones, comentarios y reacciones. Las consultas en grafos permiten analizar la influencia de un usuario dentro de una comunidad y detectar tendencias.

Sistemas de Recomendaciones: Uso de grafos para identificar patrones en las interacciones de los usuarios y sugerir contenido relevante basado en relaciones indirectas.

- Fraude Financiero: Detección de transacciones sospechosas mediante análisis de redes de conexiones y detección de patrones anómalos.
- Gestión de Redes: Optimización de rutas y relaciones en infraestructuras de telecomunicaciones para mejorar la eficiencia del tráfico de datos.

- Análisis Genómico: Uso de grafos para representar y analizar relaciones entre genes y proteínas en estudios biomédicos.
- Rutas y navegación
- Dependencias entre objetos (como paquetes, módulos, etc.)

Introducción a Neo4j

Dos formas de uso:

- **Neo4j Desktop (local)**: mayor control, requiere instalación.
- **Neo4j Aura (en la nube)**: gratuita para proyectos chicos y sin instalación.

Ambas ofrecen un entorno con **Neo4j Browser**, interfaz visual donde se escribe Cypher y se visualiza el grafo.

Introducción al lenguaje Cypher

Cypher es el lenguaje de consulta de Neo4j, diseñado específicamente para trabajar con estructuras de grafos. Su sintaxis es declarativa y permite expresar consultas de manera intuitiva, similar a SQL, pero optimizada para modelar y analizar relaciones complejas entre datos.

Conceptos básicos de Cypher

- **Nodos**: Representan entidades en el grafo.
- **Relaciones**: Conectan nodos y pueden tener direcciones y propiedades.
- **Propiedades**: Información almacenada dentro de nodos y relaciones.
- **Etiquetas**: Clasifican los nodos en distintos tipos.

Crear nodos:

```
CREATE (:Persona {nombre: "Ana", edad: 30})
CREATE (:Empresa {nombre: "TechCorp", rubro: "Software"})
```

Crear relaciones:

```
MATCH (p:Persona {nombre: "Ana"}), (e:Empresa {nombre: "TechCorp"})
CREATE (p)-[:TRABAJA_EN]->(e)
```

Consultar nodos y relaciones:

```
MATCH (p:Persona)-[:TRABAJA_EN]->(e:Empresa)
RETURN p.nombre, e.nombre
```

Consultas más complejas:

```
MATCH (p:Persona)-[:TRABAJA_EN]->(e:Empresa {rubro: "Software"})
RETURN p.nombre
```

Ejemplo práctico de uso: Red profesional de desarrolladores

Estructura sugerida:

- (:Persona {nombre, edad, stack})
- (:Empresa {nombre, rubro})
- (:Proyecto {nombre, tecnología})
- Relaciones:
    - (:Persona)-[:TRABAJA_EN]->(:Empresa)
    - (:Persona)-[:PARTICIPA_EN]->(:Proyecto)
    - (:Persona)-[:CONOCE]->(:Persona)

Consultas sugeridas:

a. ¿Qué personas trabajan en empresas del rubro “Fintech”?

```
MATCH (p:Persona)-[:TRABAJA_EN]->(e:Empresa {rubro: "Fintech"})
RETURN p.nombre
```

b. ¿Quiénes participaron en un proyecto con tecnología “React”?

```
MATCH (p:Persona)-[:PARTICIPA_EN]->(proj:Proyecto {tecnología: "React"})
RETURN p.nombre
```

c. ¿Quiénes trabajan juntos en una misma empresa?

```
MATCH (p1:Persona)-[:TRABAJA_EN]->(e)<-[:TRABAJA_EN]-(p2:Persona)
WHERE p1.nombre < p2.nombre
RETURN p1.nombre, p2.nombre, e.nombre
```

BIBLIOGRAFÍA

Needham, M., & Hodler, A. (2019). Graph Algorithms: Practical Examples in Apache Spark and Neo4j. O'Reilly Media.

Neo4j, Inc (2025) *GraphAcademy — Free, Self-Paced, Hands-on Online Training*. Disponible en: https://graphacademy.neo4j.com/

Neo4j, Inc (2025) *Neo4j documentation*. Disponible en: https://neo4j.com/docs/

Tom Geudens (2022). *Neo4j: Graph Data Modeling*. Packt Publishing.

Atribución-No Comercial-Sin Derivadas

Se permite descargar esta obra y compartirla, siempre y cuando no sea modificado y/o alterado su contenido, ni se comercialice. Referenciarlo de la siguiente manera: Universidad Tecnológica Nacional Facultad Regional Córdoba (S/D). Material para la Tecnicatura Universitaria en Programación, modalidad virtual, Córdoba, Argentina.